

JavaScript-01

Moduler

19 oktober 2024

JavaScript – utan tjafs
Författare: Acke Strömberg



För en kanin

class.js — Design Patterns

s .../TypeScript

JS class.js .../JavaScript M X

JS basic_module.js

JS revealing_module.js X

Module Pattern > JavaScript > JS class.js > ...

You, 56 seconds ago | 2 authors (Armando Peña Tamayo (Ender) and others)

Module Pattern > JavaScript > JS revealing_module.js > ...

```
1 // The Revealing Module Pattern
2 const Person = (function () {
3   // Private Properties
4   let name;
5   let lastName;
6
7   function publicSetName(_name, _lastName) {
8     name = _name;
9     lastName = _lastName;
10  }
11
12  function publicGetName() { return `${name} ${lastName}` }
13
14  /**
15   * Reveal public pointers to
16   * private functions and properties
17   */
18  return {
19    setName: publicSetName,
20    getName: publicGetName
21  }
22 })();
23
```

JavaScript git:(main) node basic_module.js

6.

JavaScript git:(main) x

Moduler

Huvudfil och type

När man skriver interaktiva webbsidor som har mer än en sida, tenderar den JavaScript (= JS) man lägger till, att svälla och bli svår att hantera. För att undvika det här problemet ska man använda sig av moduler. Det är egentligen bara ett sätt att dela in koden i olika filer så att man får lite bättre ordning på sin JavaScript. Ett sätt att dela in dem på är filer som gör specifika saker som att hämta information från ett REST API

(= Representational State Transfer Application Programming Interface) eller lägga till listelement till en lista. Man kan också dela in JS så att de är direkt kopplade till en htmlfil. Hur man gör indelningen beror lite grann på hur projektet ser ut. Men ett vanligt scenario som rekommenderas är att använda sig av en huvudfil, till exempel main.js och koppla till andra filer från den filen. Se bild 1 för exempel på hur kopplingen till en huvudfil bör se ut.

```
<script src="main.js" type="module" defer></script>
```

Bild 1 Huvudfilen på en webbsidas html-fil.

Lägg noga märke till att attributet type är tillagt. Det ska ha värdet "module" för att man ska kunna använda sig av moduler.

Export och import

För att kunna dela upp koden så behöver man exportera funktioner, variabler och andra värden till andra filer. Man har bestämt sig för att jobba huvudsakligen i main.js och hämtar in det man behöver till den filen. Det innebär att man behöver importera de delarna till main.js. Man kan antingen använda sig av **named exports** eller **default exports**. Det finns en liten skillnad mellan dessa, men huvudsakligen klarar man sig alldeles utmärkt genom att bara använda sig av named exports. När man använder sig av denna typen så sätter man bara export framför det man vill exportera. Därefter kan man importera det till den fil man vill med hjälp av klammerparenteser, eller måsvingar som man använder i svenskan. När man importerar en **named export** måste man använda samma namn som den har i den andra filen. Man kan exportera flera värden från en och samma fil när man använder den här typen av export.

```
// Filen math.js

// Vi exporterar flera olika funktioner från samma fil
export function add(a, b) {
  return a + b;
}

export function subtract(a, b) {
  return a - b;
}

// Filen main.js

// Vi importerar de specifika funktionerna med hjälp av måsvingar {}
import { add, subtract } from './math.js';

// Vi använder funktionerna
const summa = add(10, 5);
const skillnad = subtract(10, 5);

console.log(summa);    // Visar: 15
console.log(skillnad); // Visar: 5
```

Bild 2 Named export från filen math.js och en import till filen main.js.

En **default export** används när en fil har **ett** huvudsyfte eller **en** primär sak (en funktion, klass eller ett objekt) som den vill dela med sig av. Du kan döpa om den till vad du vill när du importerar.

På bild 3 kan man se hur en **default export** av samma funktion går till.

```
// Filen math.js

// En enkel funktion som vi vill ska vara filens huvudsakliga funktion
function multiply(a, b) {
  return a * b;
}

// Vi exporterar funktionen som en "default export"
export default multiply;

// Filen main.js

import multiplyTwoNumbers from './math.js';

// Nu använder du den med det nya namnet
const resultat = multiplyTwoNumbers(5, 4);
console.log(resultat); // Visar: 20
```

Bild 3 Default export från filen math.js och en import till filen main.js.

En **default export** görs genom att längst ner på den filen man vill exportera ifrån sätter export default framför. Man kan endast använda sig av en export default per fil. Till den filen man importerar så kan man importera den utan parenteser, och man har dessutom friheten att sätta ett helt valfritt namn på det man importerar. Det vill säga man måste inte använda samma namn som den hade i export-filen.